environment/02/app/workflows/creates_project.rb
```
def build
  self.project = Project.new(name: name)
  project.tasks = convert_string_to_tasks
  project.add_users(users)
  project
end
```

And the tests pass again. You can now remove the four user-related require statements and the tests still pass.

What have you gained by writing your test this way and what have you lost?

This is tricky because the loss is up-front and clear at the moment, while the potential gain is further down the road. You've lost a small piece of Rails convenience by bypassing the Rails users association in favor of a method specifically designed to be part of the public API.

You've gained the ability to think about creating projects without dealing with the complexity of users. (Many Rails conveniences work against separation of concerns in a complex program, which is a good summary of the community's argument over this kind of code.)

This particular case is too small for that to seem like much of a win (an occupational hazard of trying to show object-oriented techniques to manage complexity is that they all look like crazy overkill when applied to your typical book-example-sized problem). In a more complicated problem, separating one wildly complex piece from another wildly complex piece can make it easier to deal with each part separately.

**Prescription 51** You can use test doubles to remove test dependencies.

Removing dependencies starts to get more challenging as the underlying logic becomes more complex. For example, you might need some information about the user to create projects, most likely for access-control purposes. You might want to prevent a user from having created two projects with the same name, scoped to only the projects that particular user has created.

There are a few options for managing that complexity:

- You could just give up, add User back to CreatesProject, and just deal with the dependency. I don't want to underestimate this option. Sometimes it's the correct option, or at least the minimally complex one.
- You can create a dummy User class inside the test and require that version of User rather than the ActiveRecord version. I don't recommend this, as